A Research Report On

“SQL in E-Commerce Logistics: Inventory, Order Lifecycle, and Delivery Operations”

by

Avinash Kumar Kanojiya

Project overview

This project demonstrates my ability to model real e-commerce logistics workflows: inventory checks, purchase orders, packing, batching, routing, and delivery, using SQL-based systems. I worked with a realistic relational database schema and executed analytical SQL queries to evaluate customer order behavior, warehouse efficiency, batch performance, and delivery route metrics. The project highlights how SQL supports real-time tracking and operational decision-making in logistics environments. It showcases hands-on experience with data modeling concepts, joins, aggregations, and workflow analysis.

Inventory Workflow in the E-Commerce Platform

The inventory workflow plays a critical role in ensuring the availability of products to customers in a timely and efficient manner. In this e-commerce system, the Inventory Management System (IMS) is responsible for tracking stock levels, triggering restocking activities, and maintaining accuracy across multiple warehouse locations. The workflow outlined below captures the end-to-end inventory lifecycle — from stock evaluation to receipt and update.

1. Inventory Check and Stock Evaluation

The first step in the inventory workflow is the periodic evaluation of stock levels for each SKU (Stock Keeping Unit). This is typically done through automated SQL queries executed by the IMS or manually checked by warehouse staff.

Each SKU in the database has a predefined **reorder threshold**. This threshold indicates the minimum quantity at which new stock must be ordered to prevent a stockout. The IMS continuously monitors inventory and checks whether the current quantity of any SKU has fallen below this threshold.

Decision Logic:

- If the current stock of a SKU is **below the reorder level**, the system flags it for restocking by initiating a Purchase Order (PO).
- If the stock is **sufficient**, the process ends for that SKU, and no action is taken.

2. Purchase Order (PO) Creation

When the system identifies that stock levels are insufficient, it proceeds to create a **Purchase Order**. A PO is a formal document that requests a supplier to deliver a specified quantity of goods at agreed prices and delivery timelines.

The PO contains:

- The names and quantities of the items needed
- Agreed-upon unit prices
- Delivery date expectations
- Vendor contact details

This document not only serves as an internal authorization for procurement but also functions as a legal agreement with the supplier.

3. PO Transmission to Vendor

Once the PO is created, it is transmitted to the supplier. This can be done through multiple channels including email, web portals, or Electronic Data Interchange (EDI). The purpose of this step is to confirm:

- The required quantity of goods
- Price per unit
- Delivery timeframes

While this step may not involve complex SQL logic directly, the confirmation and communication metadata (timestamps, status, etc.) are often logged in the database.

4. Shipment and Invoice Generation by Vendor

After receiving the PO, the vendor proceeds to fulfill the request by preparing the goods for shipment. Along with the physical shipment, an **invoice** is either included in the package or sent digitally.

This invoice contains:

- The list of goods shipped
- Total cost
- Applicable taxes

- PO reference for verification

Once received, invoice data may be entered into the system to validate consistency between ordered and received goods.

5. Goods Receipt and Inspection at Warehouse

When the shipment arrives at the warehouse, staff members receive and inspect the goods. This step ensures that the received quantity and quality match the expectations defined in the PO and the invoice.

Key checks include:

- Are the quantities correct?
- Are the goods in acceptable condition?
- Do the goods match the original PO specifications?

If discrepancies are found (e.g., damaged items, quantity mismatches), they are logged for vendor follow-up and accounting adjustments.

6. Goods Receipt Note (GRN) Generation

Upon successful verification, the warehouse staff generates a **Goods Receipt Note (GRN)**. This document serves as formal proof that the ordered goods have been received. It is linked to the original PO and the invoice to maintain a traceable record for auditing and compliance.

7. Inventory Update in IMS

Following GRN generation, the Inventory Management System updates the actual stock levels in the database. The stock quantity field for each SKU is incremented based on the quantity received.

This ensures real-time visibility into what items are available for:

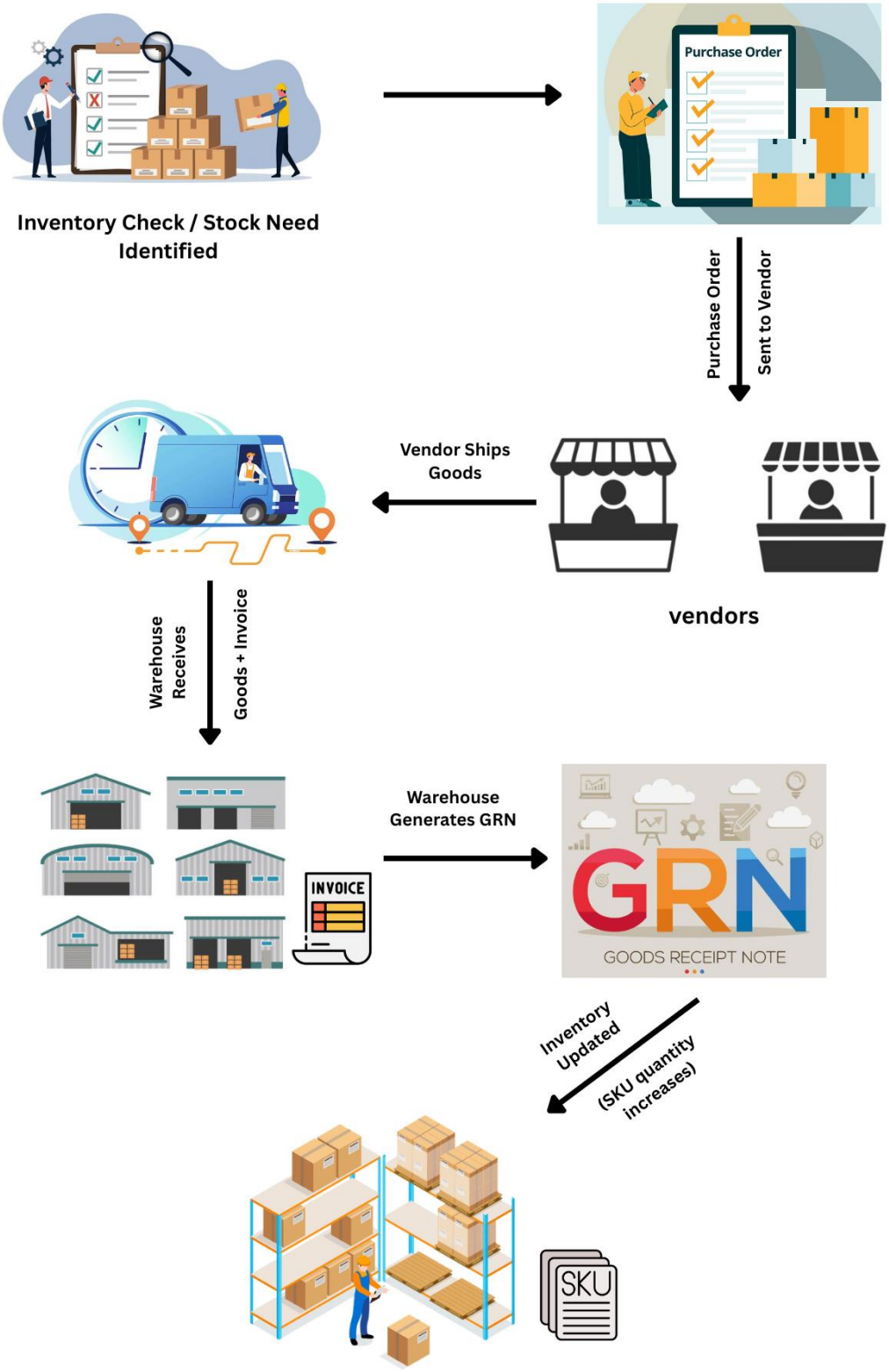
- Customer orders
- Internal operations
- Transfers between warehouses

8. Inventory Adjustments from Other Events

In addition to restocking through POs, inventory levels can change due to various operational events.

Event	Impact	SQL Action
Customer Order Placed	Decrease stock	stock -= qty
Order Return	Increase stock	stock += qty
Order Cancelled	Restore reserved stock	stock += qty
Goods Damaged	Decrease stock, record loss	stock -= qty
Lost or Pilfered Stock	Decrease stock	stock -= qty
STN (Warehouse Transfer)	Adjust both source and destination stock	stock -=/+= qty

INVENTORY WORKFLOW



Order Workflow in the E-Commerce Platform

The order workflow represents the full journey of a customer's order — from placement through dispatch, delivery, and potential return. This process is orchestrated by the Inventory Management System (IMS) and Delivery Management System (DMS), both of which rely heavily on SQL to ensure smooth execution, data integrity, and real-time tracking. The workflow is tightly integrated with inventory, warehouse operations, batch packing, and last-mile delivery.

1. Customer Order Placement

The order workflow begins when a customer places an order through the e-commerce platform. The system captures:

- Customer details
- Ordered items (SKUs and quantities)
- Payment method (e.g., COD, prepaid)
- Delivery address

The data is inserted into the **Orders** table, and a unique order ID is generated.

2. Inventory Allocation for Orders

Immediately after order creation, the IMS checks stock levels and allocates the required quantity from the available stock. This is a critical step — if stock is unavailable, the order is flagged or rejected.

Stock Allocation Logic:

- Decrease available stock by ordered quantity
- Mark items as "reserved" for this order

3. Batch Generation (Warehouse Processing)

Orders are grouped into **batches** based on rules such as:

- Destination (spoke/locality)
- Delivery hub
- Warehouse shift (morning, evening)

Once the number of orders reaches a threshold, a batch is automatically generated. This helps optimize picking, packing, and dispatch operations.

4. Picking Process

After a batch is created, **pickers** are assigned to collect items from various shelves in the warehouse. The system provides a digital list showing:

- SKU names
- Quantities
- Warehouse locations (racks, bins)

Pickers gather items and bring them to the packing area.

This process is tracked in the system using batch-related entries, often updated with timestamps to monitor picking duration.

5. Packing Process

At the packing station:

- Orders are consolidated
- Items are grouped logically (one order may be split into multiple packets)

- Each packet is given a **shipment ID** (barcode)

The packing executive marks packets as “packed” in the system. Shipment IDs are recorded and linked to their respective order IDs.

At this point, the order is visible to internal staff with full shipment details. Customers, however, can only view high-level order tracking (e.g., packed, dispatched, out for delivery).

6. Dispatch to Delivery Hub (Middle Mile)

Once packed, shipments are scanned and loaded into vehicles for transportation to the correct **delivery hub**.

Details recorded during this step include:

- Vehicle number
- Dispatch time
- Items in the shipment
- E-Waybill number (mandatory government document)

7. Receiving at Delivery Hub

Upon arrival at the hub, the delivery house executive marks the packets as "received" in the system. The delivery route (db_routes) is marked as reached, and preparation for last-mile delivery begins.

8. Last-Mile Delivery (Out for Delivery - OFD)

Orders are then sorted by **delivery routes** based on area/locality. Each delivery boy is assigned to a route with multiple orders.

- Delivery route and order list are uploaded in the system.

- The delivery boy receives packets and marks them as "Out For Delivery."

This step is managed via the db_route_mapping table, which tracks delivery personnel and order assignments.

9. Delivery Execution & Return to Origin (RTO)

The delivery boy attempts delivery at the customer's address. Orders can be:

- **Delivered** (success)
- **Marked as RTO** (Return to Origin), if the customer is unavailable or refuses delivery

Updates are made in real time using mobile apps integrated with the central database.

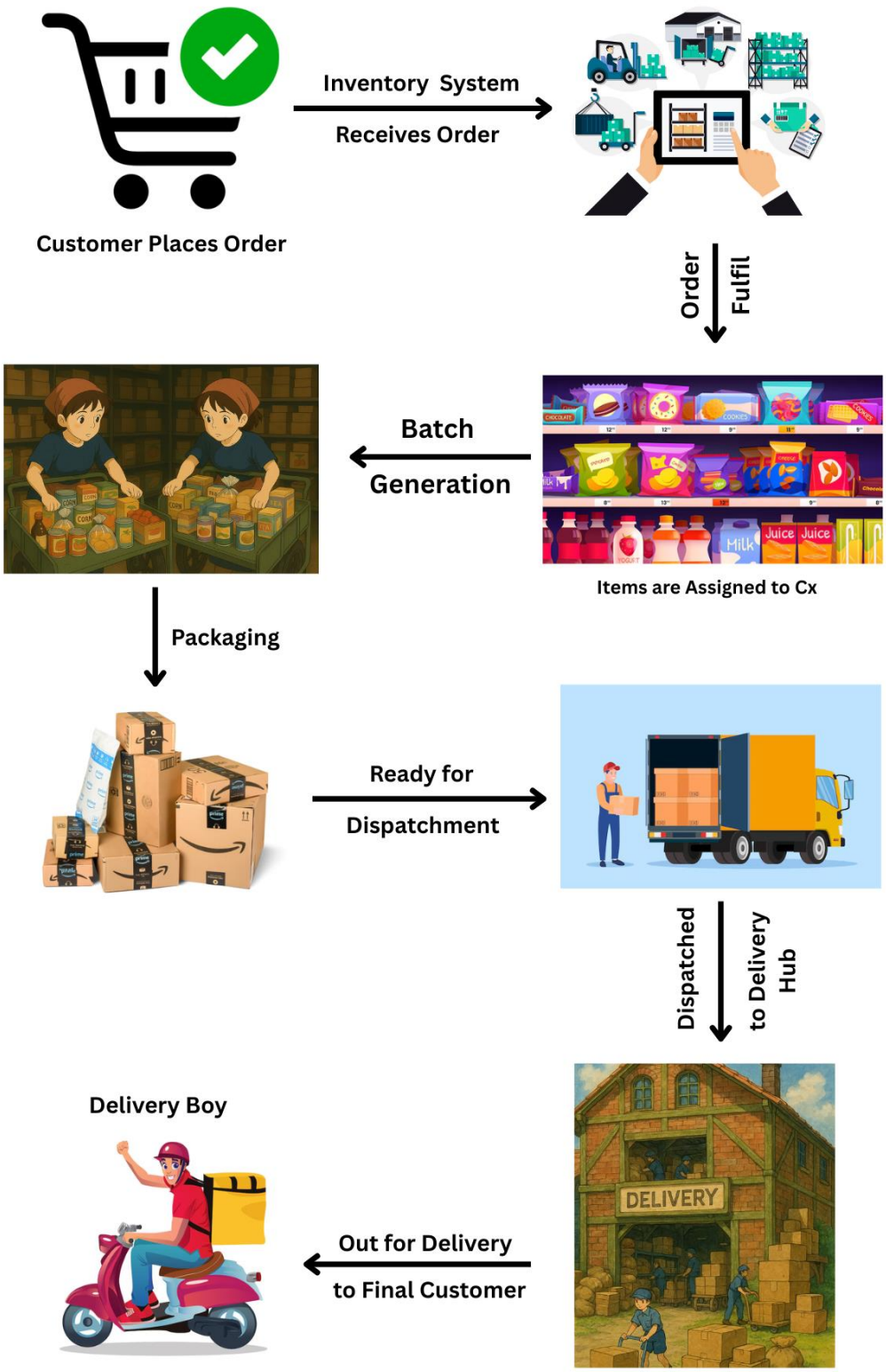
10. Reconciliation and Route Closure

At the end of the delivery run:

- The delivery boy returns to the hub
- RTOs are handed back and logged
- Cash collected is submitted
- The route is closed in the system

This ensures proper closure of financial and logistical records.

ORDER


Customer Places Order
Inventory System
Receives Order
Order
Fulfil
Items are Assigned to Cx
Batch
Generation
Packaging
Ready for
Dispatchment
Dispatched
to Delivery
Hub
DELIVERY
Out for Delivery
to Final Customer
Delivery Boy

Database Schema and Relationships

```
CREATE TABLE tbl_admin (
      admin_id BIGSERIAL PRIMARY KEY,
      admin_name TEXT NOT NULL
      );


CREATE TABLE customers (
      id BIGSERIAL PRIMARY KEY,
      name TEXT NOT NULL
      );


CREATE TABLE orders (
      order_id BIGSERIAL PRIMARY KEY,
      customer_id BIGINT,
      customer_name TEXT,
      order_date TIMESTAMP DEFAULT Now(),
      CONSTRAINT fk_customer FOREIGN KEY (customer_id) REFERENCES
customers(id)
      );


CREATE TABLE wh_batch_shifts (
      id BIGSERIAL PRIMARY KEY,
      warehouse_name TEXT NOT NULL,
      shift_name TEXT NOT NULL,
      start_time TIME NOT NULL,
      end_time TIME NOT NULL,
      created_at TIMESTAMPTZ DEFAULT Now()
      );


CREATE TABLE wh_packing_tables (
      id BIGSERIAL PRIMARY KEY,
      warehouse_name TEXT NOT NULL,
      table_number TEXT NOT NULL,
      is_disabled BOOLEAN NOT NULL DEFAULT FALSE,
```

```
	capacity BIGINT NOT NULL,
	created_by BIGINT,
	created_at TIMESTAMPTZ DEFAULT Now(),
	UNIQUE (warehouse_name,table_number),
	CONSTRAINT  fk_created_by_admin  FOREIGN  KEY  (created_by)
REFERENCES tbl_admin(admin_id)
	);


CREATE TABLE batch_summary (
	id BIGSERIAL PRIMARY KEY,
	batch_id BIGINT UNIQUE NOT NULL,
	table_number_id BIGINT NOT NULL,
	shift_id BIGINT,
	line_items_count INTEGER NOT NULL,
	packed_line_items_count INTEGER NOT NULL DEFAULT 0,
	nf_line_items_count INTEGER NOT NULL DEFAULT 0,
	spoke_name TEXT NOT NULL,
	locality TEXT,
	is_closed BOOLEAN NOT NULL DEFAULT FALSE,
	created_at TIMESTAMPTZ DEFAULT Now(),
	updated_at TIMESTAMPTZ,
	updated_at_for_pipeline TIMESTAMPTZ,
	CONSTRAINT  fk_table_number  FOREIGN  KEY  (table_number_id)
REFERENCES wh_packing_tables(id),
	CONSTRAINT  fk_shift_id  FOREIGN  KEY  (shift_id)  REFERENCES
wh_batch_shifts(id)
	);


CREATE TABLE db_routes (
	route_id BIGSERIAL PRIMARY KEY,
	route_name TEXT NOT NULL,
	delivery_date DATE NOT NULL,
	customer_id BIGINT,
	priority INTEGER DEFAULT 1,
	reached_at TIMESTAMPTZ,
	started_at TIMESTAMPTZ,
	completed_at TIMESTAMPTZ,
	missed_at TIMESTAMPTZ,
	missed_reasons TEXT [],
```

```
	missed_images TEXT [],
	completed_images TEXT [],
	pod_image TEXT,
	total_cash_collected DECIMAL(8, 2) DEFAULT 0,
	total_upi_collected DECIMAL(8, 2) DEFAULT 0,
	route_parent TEXT DEFAULT NULL,
	completed_pdf TEXT,
	spoke_name TEXT,
	updated_at TIMESTAMPTZ,
	updated_at_for_pipeline TIMESTAMPTZ,
	created_at TIMESTAMPTZ DEFAULT Now(),
	misroute BOOLEAN DEFAULT FALSE,
	misroute_at TIMESTAMPTZ,
	misroute_by BIGINT,
	frozen BOOLEAN DEFAULT FALSE,
	tp_priority BIGINT DEFAULT 1,
	should_collect_cash BOOLEAN,
	dispatched_at TIMESTAMPTZ,
	picking_started_at TIMESTAMPTZ,
	CONSTRAINT	fk_customer_id	FOREIGN	KEY	(customer_id)
REFERENCES customers(id),
	CONSTRAINT	fk_misroute_by	FOREIGN	KEY	(misroute_by)
REFERENCES tbl_admin(admin_id)
	);


CREATE TABLE db_route_mapping (
	id BIGSERIAL PRIMARY KEY,
	route_id BIGINT,
	delivery_boy_id BIGINT NOT NULL,
	delivery_boy_username TEXT NOT NULL,
	delivery_boy_name TEXT NOT NULL,
	delivery_boy_phone TEXT NOT NULL,
	delivery_boy_alternate_phone TEXT,
	delivery_date DATE NOT NULL,
	priority INTEGER DEFAULT 1,
	upi_ok BOOLEAN,
	cash_ok BOOLEAN,
	upi_mismatch_reason TEXT,
	cash_mismatch_reason TEXT,
	vehicle_number TEXT,
```

```
        driver_name TEXT,
        driver_number TEXT,
        trip_start_reading INTEGER,
        trip_stop_reading INTEGER,
        vehicle_cost DECIMAL(8, 2),
        created_at TIMESTAMPTZ DEFAULT Now(),
        collected_cod DECIMAL(8, 2) DEFAULT 0,
        collected_upi DECIMAL(8, 2) DEFAULT 0,
        spoke_name TEXT,
        updated_at_for_pipeline TIMESTAMPTZ,
        CONSTRAINT fk_route FOREIGN KEY (route_id) REFERENCES
db_routes(route_id),
        CONSTRAINT fk_delivery_boy FOREIGN KEY (delivery_boy_id)
REFERENCES tbl_admin(admin_id)
        );
```

Foreign Key References (Table Relationships)

orders.customer_id → customers.id

- Each order is placed by a customer.

wh_packing_tables.created_by → tbl_admin.admin_id

- A warehouse packing table is created by an admin (e.g., warehouse manager).

batch_summary.table_number_id → wh_packing_tables.id
batch_summary.shift_id → wh_batch_shifts.id

- A batch is associated with a specific packing table and a shift in the warehouse.

db_routes.customer_id → customers.id

- A delivery route is linked to a customer.

db_routes.misroute_by → tbl_admin.admin_id

- If a route is misrouted, it's marked by an admin.

db_route_mapping.route_id → db_routes.route_id
db_route_mapping.delivery_boy_id → tbl_admin.admin_id

- A delivery route is assigned to a delivery boy, who is also an admin in the system.

Data Analysis Queries

1. Total Orders Placed per Customer

```
SELECT c.id AS customer_id,
       c.name AS customer_name,
       COUNT(o.order_id) AS total_orders
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
GROUP BY c.id,
ORDER BY total_orders DESC
LIMIT 10;
```

What this Query Does:

- Counts the number of orders per customer.
- Helps identify high-frequency customers.
- Shows top 10 customers by total orders.

Sample Result Table:

customer_id	customer_name	total_orders
205	Alice Johnson	18
182	Michael Smith	15
219	Raj Patel	14
198	Linda Martinez	12
211	James Anderson	11
176	Emily Thompson	10
207	Sophia Kim	9
202	David Wilson	8
230	Mohammed Ali	8
187	Olivia Chen	7

2. Batches with the Most Unpacked Items

```
SELECT batch_id,
       line_items_count,
       packed_line_items_count,
       (line_items_count - packed_line_items_count) AS items_pending,
       is_closed,
       created_at
FROM batch_summary
WHERE is_closed = FALSE
ORDER BY items_pending DESC
LIMIT 10;
```

What this Query Does:

- Shows open batches and how many items are yet to be packed.
- Helps in identifying bottlenecks in packing.

Sample Result Table:

batch_id	line_items_count	packed_line_items_count	items_pending	is_closed	created_at
1009	200	120	80	FALSE	2025-07-10 14:23:00
1012	150	75	75	FALSE	2025-07-10 16:05:00
1011	180	110	70	FALSE	2025-07-10 15:00:00
1003	120	60	60	FALSE	2025-07-09 11:30:00
1006	100	50	50	FALSE	2025-07-09 16:45:00
1001	90	45	45	FALSE	2025-07-08 10:00:00
1010	70	35	35	FALSE	2025-07-10 14:45:00
1004	60	30	30	FALSE	2025-07-09 12:15:00
1007	50	25	25	FALSE	2025-07-09 17:30:00
1002	40	20	20	FALSE	2025-07-08 11:15:00

3. Delivery Route Performance Matrix

```
SELECT r.route_id,
       COUNT(ro.order_id) AS total_orders,
       r.total_cash_collected,
       r.total_upi_collected,
       r.completed_at
FROM db_routes r
LEFT JOIN route_orders ro ON r.route_id = ro.route_id
GROUP BY r.route_id,
         r.total_cash_collected,
         r.total_upi_collected,
         r.completed_at
ORDER BY r.completed_at DESC
LIMIT 10;
```

What this Query Does:

- Counts the number of orders associated with each delivery route.
- Summarizes total cash and UPI payments collected per route.
- Helps assess financial performance and completion status of recent routes.

Sample Result Table:

route_id	total_orders	total_cash_collected	total_upi_collected	completed_at
3108	45	12,500.00	6,300.00	2025-07-11 16:20:00
3103	42	11,800.00	5,900.00	2025-07-11 15:55:00
3097	39	10,400.00	5,600.00	2025-07-11 15:40:00
3088	40	11,000.00	5,300.00	2025-07-11 15:15:00
3075	38	9,950.00	4,800.00	2025-07-10 15:50:00
3069	37	9,700.00	4,500.00	2025-07-10 15:20:00
3056	36	9,200.00	4,200.00	2025-07-10 14:45:00
3042	34	8,600.00	4,100.00	2025-07-09 15:30:00
3038	33	8,400.00	3,950.00	2025-07-09 15:05:00

4. Total Deliveries by Each Delivery Boy

```
SELECT a.admin_name AS delivery_boy,
       COUNT(drm.route_id) AS total_deliveries
FROM db_route_mapping drm
JOIN tbl_admin a ON drm.delivery_boy_id = a.admin_id
GROUP BY a.admin_name
ORDER BY total_deliveries DESC;
```

What this Query Does:

- Counts how many delivery routes each delivery boy has handled.
- Lists delivery boys ranked by total deliveries (most to least).

Sample Result Table:

delivery_boy	total_deliveries
Arjun Mehta	58
Ramesh Kumar	54
Neeraj Singh	49
Pooja Verma	46
Ankit Sharma	44
Priya Nair	42
Deepak Joshi	39
Kavita Rao	36
Rahul Desai	34
Sneha Patil	32

5. Average Orders per Customer

```
SELECT c.id AS customer_id,
       c.name AS customer_name,
       COUNT(o.order_id) AS total_orders,
       ROUND(COUNT(o.order_id) * 1.0 / (DATE_PART('day', MAX(o.order_date)
- MIN(o.order_date)) + 1), 2) AS avg_orders_per_day
FROM customers c
JOIN orders o ON c.id = o.customer_id
GROUP BY c.id,
         c.name
ORDER BY avg_orders_per_day DESC
LIMIT 10;
```

What this Query Does:

- Calculates total and average daily orders for each customer based on their order history.
- Ranks the top 10 customers by average orders per day.
- Helps identify the most consistently active customers.

Sample Result Table:

customer_id	customer_name	total_orders	avg_orders_per_day
104	FreshMart	95	3.17
109	Green Basket	88	2.93
117	Organic Avenue	78	2.79
125	QuickGrocer	105	2.68
132	Daily Needs	70	2.58
141	TownFresh	81	2.44
149	Farm Direct	66	2.36
155	Urban Cart	74	2.31
162	Fresh & Fast	61	2.18
170	LocalHarvest	69	2.13

6. Inactive Delivery Boys (No Routes in Last 7 Days)

```
SELECT a.admin_id,
       a.admin_name
FROM tbl_admin a
LEFT JOIN db_route_mapping drm ON a.admin_id = drm.delivery_boy_id
AND drm.delivery_date >= CURRENT_DATE - INTERVAL '7 days'
WHERE drm.id IS NULL;
```

What this Query Does:

- Finds admins/delivery boys who haven't been assigned a route in the last week.
- Helpful for workforce planning or investigating absenteeism.

Sample Result Table:

admin_id	admin_name
201	Rakesh Verma
204	Nidhi Shah
209	Aman Kapoor
213	Kavya Nair
217	Imran Qureshi
220	Suman Gupta
223	Pooja Bansal
225	Dev Singh
228	Tanya Mehra
230	Rohit Sinha

7. Average Delivery Completion Time

```
SELECT AVG(EXTRACT(EPOCH
                   FROM   (completed_at   -   started_at))   /   60)   AS
avg_delivery_minutes
FROM db_routes
WHERE completed_at IS NOT NULL
  AND started_at IS NOT NULL;
```

What this Query Does:

- Calculates average time (in minutes) between started_at and completed_at for deliveries.
- Useful for SLA tracking.

Sample Result Table:

avg_delivery_minutes
97.3

FINDINGS

1. SQL supports the entire logistics workflow — from when a customer places an order to final delivery. It helps manage inventory checks, order status, packing progress, and delivery updates in a structured and reliable way.

2. Batch creation and packing operations are efficiently handled using SQL. The system groups orders into batches once a threshold is reached and assigns them to packing tables. SQL tracks how many items were packed, not found, or still pending, which helps monitor warehouse performance.

3. Delivery route and mapping data stored in the system helps track how many orders are delivered per route, who handled the delivery, and how much cash or UPI was collected. This is useful for both operational planning and financial tracking.

4. SQL queries can identify underused resources such as inactive packing tables or delivery boys who haven't handled routes recently. This helps improve resource planning and workforce management.

5. Payment mismatches or issues (e.g., cash not collected or UPI failed) can be flagged and reviewed using specific fields in the database like cash_ok, upi_ok, and mismatch reasons. This improves accountability and reduces financial errors.

6. SQL-based aggregation queries support operational audits such as identifying inactive delivery boys, missed delivery causes, and shift-wise packing outputs.

7. Average-based metrics such as average batch size, average delivery time, or average orders per customer give useful insights that can be used to improve efficiency and customer service.

CONCLUSION

This research project explored how SQL is used in managing logistics operations within an e-commerce platform, focusing on processes such as inventory control, order handling, batch creation, and final delivery. Through the design and analysis of a realistic database schema, the study showed how SQL enables accurate, real-time tracking and management of complex logistics workflows.

The system designed in this project reflects actual operations observed in modern e-commerce, especially those serving tier-2 and tier-3 cities in India. SQL was used not only to store and retrieve data but also to monitor performance, track payments, generate summaries, and support operational decisions.

By simulating order-to-delivery workflows and executing meaningful SQL queries, the study demonstrated how structured databases contribute to better resource utilization, faster order fulfillment, and improved delivery tracking. Features like cash/UPI tracking, batch status, and delivery route monitoring show the depth of control and analysis made possible with SQL.

Given these observations, the evidence supports that there is a relationship between SQL implementation and improved logistics efficiency. In conclusion, adopting SQL-based database systems in e-commerce logistics not only strengthens data accuracy and operational control but also enables faster, more reliable, and customer-focused order fulfillment.

SUGGESTIONS

1. **Add indexes on frequently queried columns** such as customer_id, route_id, shift_id, and completed_at to improve the speed and efficiency of joins, filters, and report generation across large volumes of logistics data.
2. **Implement smart batch triggers** that consider both order volume and time elapsed since the last batch, allowing the system to generate batches either when a threshold is met or after a certain delay—this prevents long wait times during low-demand periods and keeps operations flexible.
3. **Monitor packing table usage regularly:** Use SQL queries to track inactive or underused packing tables. This can help warehouse managers assign staff more efficiently and make better use of available space.
4. **Add order status tracking fields:** Introducing clear order status values (like "Pending," "Packed," "Dispatched," "Delivered") would make it easier to track order progress through each stage of the logistics workflow.
5. **Include delivery performance metrics in dashboards:** Regularly review route and delivery boy performance using SQL-based summaries. This can help identify top performers, missed deliveries, and areas that need improvement.
6. **Flag financial mismatches in real-time:** Use SQL to automatically flag cash or UPI mismatch cases so that they can be reviewed quickly. This reduces errors and improves accountability.
7. **Use averages and trends to guide planning:** Metrics like average batch size, average orders per customer, or average delivery time can help the company plan staffing, route design, and order handling better.
8. **Implement role-based access in the system:** Limit data access using admin roles (warehouse staff, delivery boys, supervisors) to ensure users only view what is necessary for their work.